

JPA Entity Mappings

Chapter 04 - JPA & Hibernate

Sub-Chapter 04.03

@OneToOne	One entity relates to exactly one other entity
@OneToMany	One entity relates to many child entities
@ManyToMany	Both sides relate to many of each other
Cascade Types	Propagate operations to related entities
@Embedded	Embed value objects inside an entity table
Inheritance	Map class hierarchies to database tables

1 @OneToOne Mapping

Person has exactly one Passport

A @OneToOne mapping links two entity classes where each instance on one side corresponds to exactly one instance on the other side. A foreign key is stored in one of the tables (the owning side). JPA handles this automatically when you annotate the relationship field with @OneToOne.

Java Example

```
@Entity
public class Person {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "passport_id")
    private Passport passport;
}

@Entity
public class Passport {
    @Id @GeneratedValue
    private Long id;
    private String passportNo;

    @OneToOne(mappedBy = "passport")
    private Person person;
}
```

Diagram

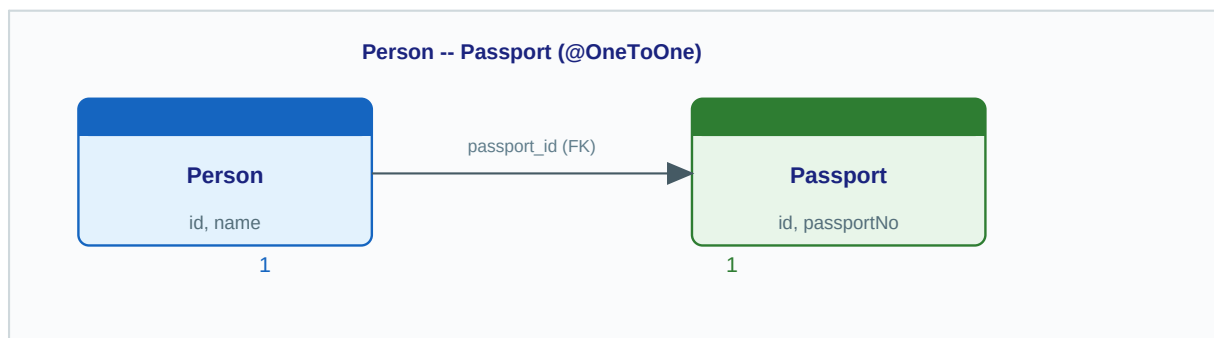


Figure 1.1 – OneToOne: Person owns FK to Passport

Key Rules

- The side with @JoinColumn is the OWNING side and holds the foreign key column.
- Use mappedBy on the INVERSE side to avoid a second FK column being created.
- CascadeType.ALL on @OneToOne means persist/remove of Person cascades to Passport.
- FetchType.LAZY is recommended for @OneToOne to avoid unnecessary joins.
- Both entities must be annotated with @Entity and have a primary key (@Id).

Q1. What annotation marks the owning side of a @OneToOne mapping?

@JoinColumn marks the owning side. The table of the owning entity stores the foreign key column pointing to the related entity.

Q2. What does mappedBy do in @OneToOne?

mappedBy tells JPA that the OTHER side owns the relationship and holds the FK. It prevents JPA from creating a duplicate FK column.

Q3. Can both sides be the owning side in @OneToOne?

No. Exactly one side must own the relationship. Only the owning side has @JoinColumn; the inverse side uses mappedBy.

Q4. What happens if you forget mappedBy on the inverse side?

JPA will create two FK columns — one in each table — leading to data inconsistency and a schema that is harder to maintain.

Q5. When would you use @OneToOne over embedding an @Embeddable?

Use @OneToOne when the related object has its own lifecycle, its own ID, and can exist independently (e.g., Passport can outlive Person).

2 @OneToMany / @ManyToOne

Customer has many Orders

This is the most common JPA relationship. One entity (Customer) is linked to many instances of another entity (Order). The child side (Order) holds a foreign key back to the parent (Customer). Use @ManyToOne on the child field and @OneToMany on the parent collection.

Java Example

```
@Entity
public class Customer {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @OneToMany(mappedBy = "customer", cascade = CascadeType.ALL)
    private List<Order> orders = new ArrayList<>();
}

@Entity
public class Order {
    @Id @GeneratedValue
    private Long id;
    private double total;

    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

Diagram

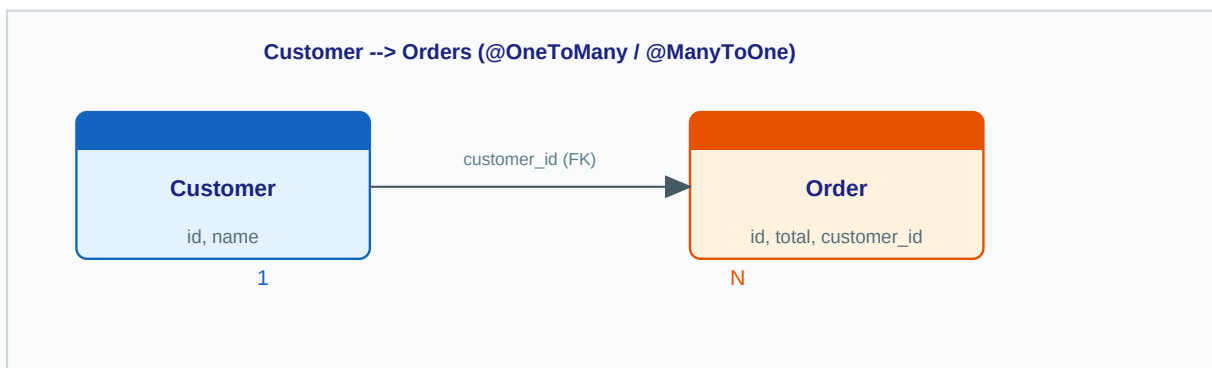


Figure 2.1 – OneToMany: FK `customer_id` lives in the Order table

Key Rules

- @ManyToOne is ALWAYS the owning side; @JoinColumn lives on the @ManyToOne field.
- The @OneToMany side uses mappedBy referencing the field name in the child entity.
- Without mappedBy, JPA creates an extra join table — usually not what you want.
- Initialize collections (`new ArrayList<>()`) to avoid NullPointerException.
- Prefer FetchType.LAZY on @OneToMany — loading all orders eagerly is expensive.

Q1. Which side holds the foreign key in a OneToMany relationship?

The MANY side (child) holds the FK. In the Customer-Order example, the Order table has a `customer_id` column.

Q2. What is the difference between @OneToMany and @ManyToOne?

@OneToMany is on the parent (one Customer has a list of orders). @ManyToOne is on the child (many Orders point to one Customer).

Q3. Why should you use mappedBy on @OneToMany?

Without mappedBy, JPA creates an intermediate join table. With mappedBy, JPA reuses the FK in the child table — simpler and faster.

Q4. What is the default fetch type for @OneToMany?

LAZY. JPA does not load the collection until you access it. This is the recommended default to avoid N+1 query problems.

Q5. How do you add an Order to a Customer bidirectionally?

Set both sides: `order.setCustomer(customer)` and `customer.getOrders().add(order)`. Keeping both sides in sync avoids stale data.

3 @ManyToMany with @JoinTable

Student enrolls in many Courses

A @ManyToMany mapping means each entity on both sides can relate to multiple instances of the other. JPA requires a join table (also called a junction or bridge table) that holds the foreign keys of both sides. You define this join table using the @JoinTable annotation.

Java Example

```
@Entity
public class Student {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses = new ArrayList<>();
}

@Entity
public class Course {
    @Id @GeneratedValue
    private Long id;
    private String title;

    @ManyToMany(mappedBy = "courses")
    private List<Student> students = new ArrayList<>();
}
```

Diagram

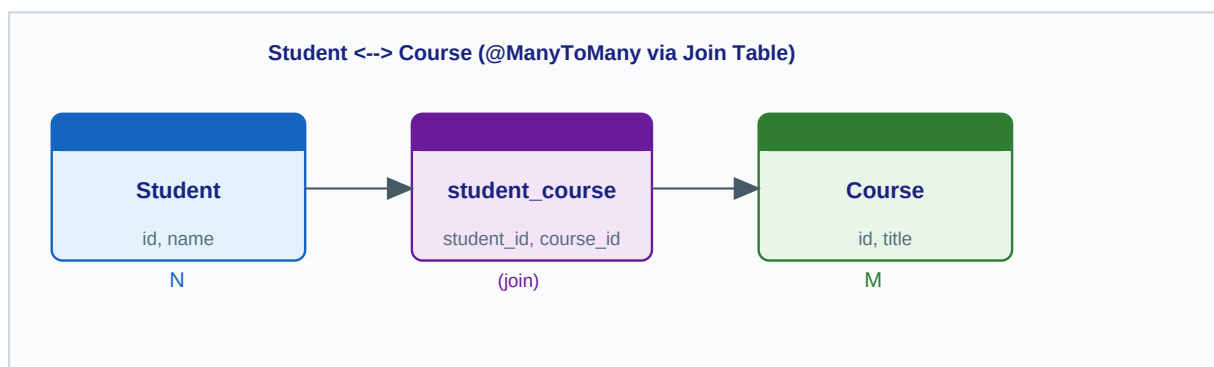


Figure 3.1 – ManyToMany: join table student_course links both entities

Key Rules

- The owning side (Student) defines @JoinTable with joinColumns and inverseJoinColumns.
- The inverse side (Course) uses mappedBy referring to the owning side's field name.
- JPA auto-creates the join table; you do NOT create a separate entity for it (unless you need extra columns).
- If you need extra columns (e.g., enrollment_date) in the join table, create an @Entity for it and use two @ManyToOne relationships.

- Default fetch for @ManyToMany is LAZY — always prefer this to avoid loading huge collections.

Q1. What is a join table in @ManyToMany?

A join table is an intermediary database table that stores pairs of foreign keys — one pointing to each side of the ManyToMany relationship.

Q2. Which side defines @JoinTable?

The owning side defines @JoinTable. The inverse side uses mappedBy. In the example, Student is the owner.

Q3. What annotation is used to name the join table and its columns?

@JoinTable(name=..., joinColumns=..., inverseJoinColumns=...) on the owning side's collection field.

Q4. When should you convert a @ManyToMany into two @ManyToOne relationships?

When the join table needs additional columns (like enrolledDate or grade), you must model it as a separate @Entity with two @ManyToOne fields.

Q5. Does JPA create the join table automatically?

Yes. If schema auto-creation is enabled (hbm2ddl.auto=update or create), JPA/Hibernate will generate the join table automatically.

4 Cascade Types

PERSIST, MERGE, REMOVE, ALL

Cascading determines what happens to the related (child) entity when an operation is performed on the parent entity. For example, if you persist a Customer, should its Orders also be persisted automatically? You control this with the cascade attribute on the relationship annotation.

Java Example

```
@Entity
public class Customer {
    @Id @GeneratedValue
    private Long id;

    // PERSIST: saving Customer also saves Orders
    // REMOVE: deleting Customer also deletes Orders
    @OneToMany(mappedBy = "customer",
        cascade = {CascadeType.PERSIST, CascadeType.REMOVE})
    private List<Order> orders = new ArrayList<>();
}

// CascadeType.ALL is shorthand for all cascade types:
@OneToMany(mappedBy = "customer", cascade = CascadeType.ALL)
private List<Order> orders;
```

Diagram

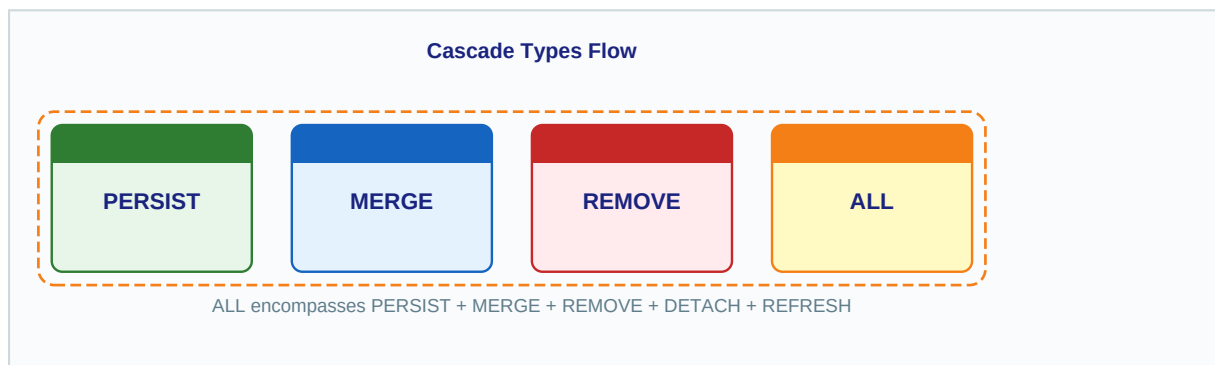


Figure 4.1 – Cascade types and what operations they propagate

Cascade Type Reference

CascadeType	Operation Propagated	Common Use Case
PERSIST	EntityManager.persist()	Auto-save child when parent is saved
MERGE	EntityManager.merge()	Auto-update child when parent is merged
REMOVE	EntityManager.remove()	Auto-delete child when parent is deleted
REFRESH	EntityManager.refresh()	Reload child from DB when parent is refreshed

DETACH	EntityManager.detach()	Detach child when parent is detached
ALL	All of the above	Use with care — covers every operation

Key Rules

- CascadeType.ALL is equivalent to listing PERSIST, MERGE, REMOVE, REFRESH, and DETACH together.
- REMOVE cascade can be dangerous — deleting a parent deletes ALL its children permanently.
- Do not use CascadeType.REMOVE when children can be shared (e.g., @ManyToMany).
- Cascade applies to JPA operations only; it does NOT set ON DELETE CASCADE in SQL (use @OnDelete for that).
- Specify only the cascades you actually need; avoid ALL unless the child has no independent lifecycle.

Q1. What does CascadeType.PERSIST do?

When you call entityManager.persist(parent), JPA automatically persists all child entities linked with CascadeType.PERSIST — no need to persist each child manually.

Q2. Is CascadeType.ALL always safe to use?

No. CascadeType.ALL includes REMOVE, which deletes children when the parent is deleted. This can cause unintended data loss if children have their own lifecycle.

Q3. What is the difference between JPA cascade and SQL ON DELETE CASCADE?

JPA cascade operates at the Java/ORM level. SQL ON DELETE CASCADE is enforced by the database engine. They are independent and both can be active at the same time.

Q4. When would you use CascadeType.MERGE?

When you detach an entity, modify it, and then re-attach it with merge(), CascadeType.MERGE ensures the child entities are also merged back into the persistence context.

Q5. Can you combine multiple cascade types?

Yes. Use cascade = {CascadeType.PERSIST, CascadeType.MERGE} to apply multiple types selectively, instead of using ALL.

5 @Embedded / @Embeddable

Address embedded inside Employee

Sometimes a group of related fields (like street, city, zip) belongs together logically but should live in the same database table as the owning entity. JPA lets you model this with `@Embeddable` (on the value object) and `@Embedded` (on the field inside the entity). No separate table is created.

Java Example

```
@Embeddable
public class Address {
    private String street;
    private String city;
    private String zip;
}

@Entity
public class Employee {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private double salary;

    @Embedded
    private Address address;
}

// DB table: EMPLOYEE(id, name, salary, street, city, zip)
```

Diagram

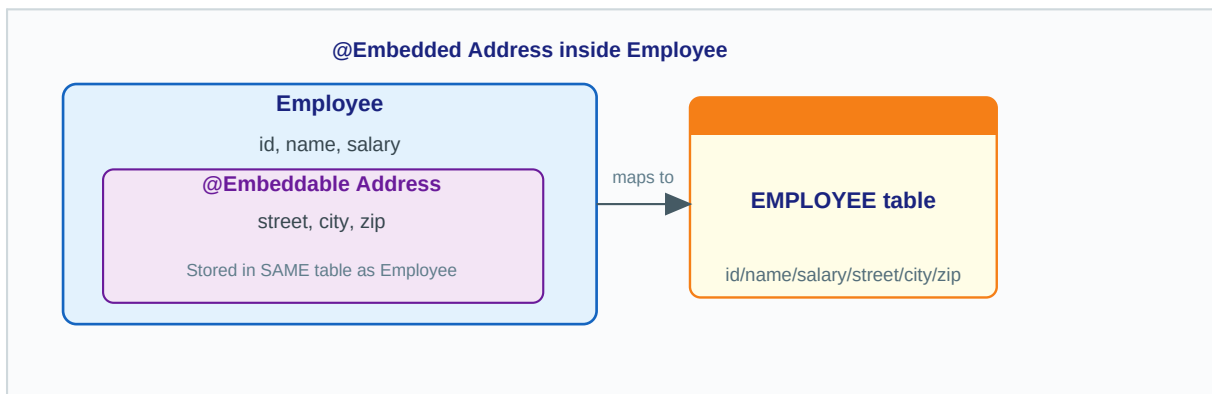


Figure 5.1 – Address is embedded: its columns live in the EMPLOYEE table

Key Rules

- `@Embeddable` classes have NO `@Id` and NO separate table — their columns merge into the owner's table.
- You can reuse the same `@Embeddable` in multiple entities (e.g., Address in Employee and Customer).
- Use `@AttributeOverride` to rename columns when the same `@Embeddable` is reused in one entity twice.
- `@Embedded` objects are not entities — they cannot be queried independently and have no lifecycle.
- If the embedded object is null, JPA stores NULL for all its columns — handle this in your code.

Q1. What is the key difference between `@Embedded` and `@OneToOne`?

@Embedded stores the data in the SAME table as the owning entity (no FK). @OneToOne uses a separate table with a foreign key between them.

Q2. Does @Embeddable need a primary key?

No. @Embeddable classes have no @Id. They are value objects whose columns are flattened into the owning entity's table.

Q3. How do you reuse an @Embeddable in the same entity twice?

Use @AttributeOverride to rename the column mappings. For example, homeAddress and workAddress both use Address, but their columns must have distinct names.

Q4. Can you query by fields inside an @Embeddable in JPQL?

Yes. Use dot notation: SELECT e FROM Employee e WHERE e.address.city = 'Delhi'. JPA translates this to a WHERE clause on the correct column.

Q5. What happens to embedded columns if the embedded object is null?

JPA stores NULL for every column of the embedded object. No exception is thrown, but you should guard against NullPointerException in your Java code.

6 Inheritance Mapping

SINGLE_TABLE, TABLE_PER_CLASS, JOINED

When you have a Java class hierarchy (e.g., Vehicle -> Car, Truck) and want to persist it to a database, JPA offers three strategies. Each strategy trades off between query performance, schema normalization, and simplicity. You choose a strategy with `@Inheritance` on the parent entity.

Java Example

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Vehicle {
    @Id @GeneratedValue
    private Long id;
    private String brand;
}

@Entity
public class Car extends Vehicle {
    private int doors;
}

@Entity
public class Truck extends Vehicle {
    private double payload;
}

// Change InheritanceType to SINGLE_TABLE or TABLE_PER_CLASS as needed.
```

Diagram

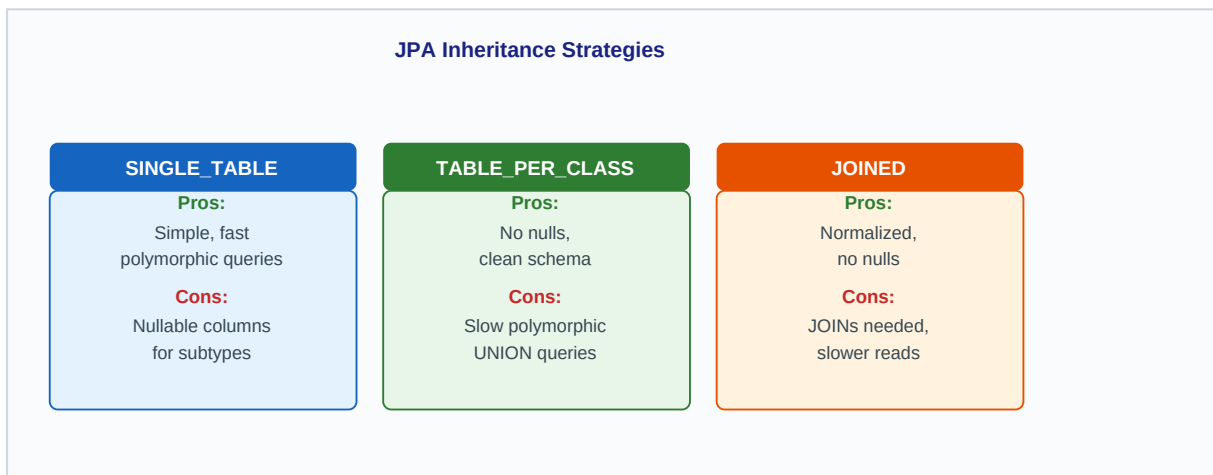


Figure 6.1 – Three-column comparison of JPA inheritance strategies

Strategy Comparison Table

Strategy	Tables Created	Pros	Cons
SINGLE_TABLE	1 (all columns in one table)	Fastest queries, simple schema	Many nullable columns, no NOT NULL constraints on subtype fields
TABLE_PER_CLASS	1 per class (no shared table)	No nulls, clean per-class table	UNION needed for polymorphic queries; slow with many subtypes

JOINED	1 parent + 1 per subtype (with FK)	Normalized, no nulls, clean design	JOIN required for every query; more complex SQL
--------	------------------------------------	------------------------------------	---

Key Rules

- SINGLE_TABLE is the JPA default strategy if you omit @Inheritance.
- JOINED is the most normalized strategy but requires a JOIN for every entity load.
- TABLE_PER_CLASS makes polymorphic queries very slow (UNION ALL across all subtype tables).
- Use @DiscriminatorColumn/@DiscriminatorValue with SINGLE_TABLE to differentiate subtype rows.
- For simple hierarchies with few subtypes, SINGLE_TABLE offers the best read performance.

Q1. What is the default JPA inheritance strategy?

SINGLE_TABLE. If you annotate the parent with @Inheritance without specifying a strategy, JPA uses SINGLE_TABLE by default.

Q2. When would you prefer JOINED over SINGLE_TABLE?

When you need a normalized schema without nullable columns for subtypes, and you can tolerate the overhead of JOIN queries on every entity load.

Q3. What SQL operation does TABLE_PER_CLASS require for polymorphic queries?

UNION ALL. JPA must query every subtype table and combine results, which is slow when there are many subtypes.

Q4. What is a discriminator column?

A special column (e.g., DTYPE) in the SINGLE_TABLE strategy that stores a value identifying which subtype each row represents, e.g., 'Car' or 'Truck'.

Q5. Can you mix inheritance strategies in the same hierarchy?

No. JPA requires a single strategy for an entire inheritance hierarchy. You cannot use JOINED for one branch and SINGLE_TABLE for another.